

Practical Points on Chunking Data

When you have a document like a blog post, article, or book chapter, the text is **continuous** — sentences flow into paragraphs, ideas build on each other linearly.

Standard text chunking (splitting by tokens/sentences/paragraphs with overlap) works well

But it **breaks down** for structured and visual data types

1. Tabular Data (Tables, CSVs, Spreadsheets)

How will you chunk it ???

The problem:

If you chunk a 500-row CSV by raw character count, you'll cut rows in half, lose the header row after the first chunk, and destroy the relationship between columns.

The solution — Table-Aware Chunking:

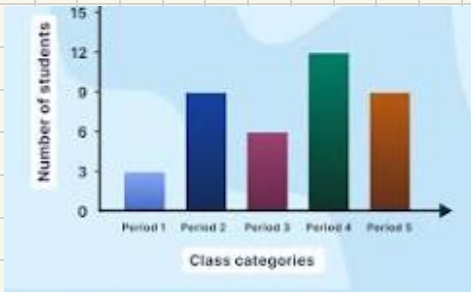
- **Split into sub-tables of ~20-30 rows** (small enough to fit in an embedding window).
- **Prepend the column headers to every sub-table** so each chunk is self-contained.
- **Include table metadata** (table name, source file, sheet name, description) as a text preamble in every chunk.

Images

An embedding model (like OpenAI's text-embedding-3-small) only understands **text**. You can't feed it a JPEG. So how do you make an image searchable in a RAG pipeline?

Option A: Captioning (Lossy)

What it means: You use a vision model (GPT-4o, Claude, etc.) to **describe the image in words**, then embed that text description.



You send it to a vision model and it returns:

"A bar chart showing quarterly revenue for 2024. Q1: \$10 million, Q2: \$12 million, Q3: \$15 million, Q4: \$18 million. Revenue shows a steady upward trend across all four quarters."

Now you embed **this text** and store it in your vector DB.

Option B: Multimodal Embedding (Lossless)

Use a **multimodal embedding model** (e.g., CLIP, Nomic Embed Vision, Cohere multimodal) that directly maps images into the same vector space as text.

Other Tricky Data Types

- **JSON DATA** --> Flatten to key-value paths: `user.address.city = "Boston"`
- **Code** --> Chunk by function/class boundaries, not token count
- **Audio** --> Transcribe (Whisper), then chunk the transcript

1 After American Airlines neared a deal in 2011 to buy jets from Airbus, Boeing quickly decided to build the 737 Max, above. BOEY WASSON/REUTERS

2 A Jet Born of a Frantic Race to Outdo a Rival

3 This article is by David Gelles, Natalie Kitroeff, Jack Nicas and Rebecca R. Ruiz.

4 Boeing faced an unthinkable defection in the spring of 2011. American Airlines, an exclusive Boeing customer for more than a decade, was ready to place an order for hundreds of new, fuel-efficient jets from the world's other major aircraft manufacturer, Airbus.

5 The chief executive of American called Boeing's leader, W. James McNerney Jr., to say a deal was close. If Boeing wanted the business, it would need to move aggressively, the airline executive, Gerard Arpey, told Mr. McNerney.

6 To win over American, Boeing ditched the idea of developing a new passenger plane, which would take a decade. Instead, it decided to update its workhorse 737, promising the plane would be done in six years.

The 737 Max was born roughly three months later.

The competitive pressure to build the jet — which permeated the entire design and development — now threatens the reputation and profits of Boeing, after two deadly crashes of the 737 Max in less than five months. Prosecutors and regulators are investigating whether the effort to design, produce and certify the Max was rushed, leading Boeing to miss crucial safety risks.

7 and to underplay the need for pilot training. While investigators are still trying to determine the cause of the crash in Ethiopia this month and one in Indonesia in October, they are focused on a newly installed piece of software designed to avoid stalls. The software was meant to compensate for bigger, more fuel-efficient engines and ensure the plane flew the same way as an earlier version.

8 Months behind Airbus, Boeing had to play catch-up. The pace of the work on the 737 Max was frenetic, according to current and former employees who spoke with The New York Times. Some spoke on the condition.

Continued on Page 16

Layout Detection , Reading Order Everything is important

<https://github.com/datalab-to/surya>

Vector Databases - How do they Work ??

Imagine you have 10,000 documents and someone asks:

"How do I reset my password?"

A normal database (like SQL) searches by **exact matches** of words.

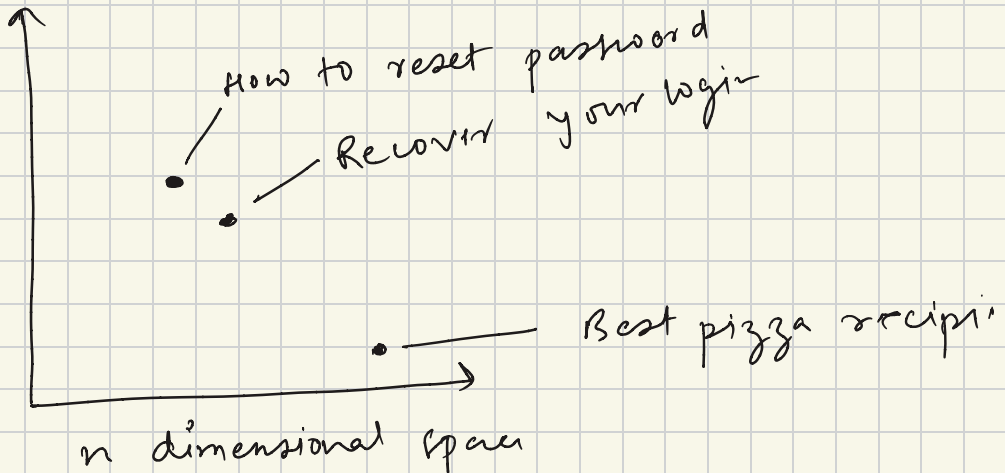
Even if one of your document has --> *"recover your login credentials"* -->
Database will miss it completely

A **vector database** is built to search by **meaning**, not by exact words.

2. The core idea: turning meaning into numbers

Convert every piece of text (or image, audio, etc.) into a list of numbers that captures its meaning.

<i>"How to reset password"</i>	→ <i>[0.91, 0.12, 0.88, 0.05, ...]</i>
<i>"Recover your login"</i>	→ <i>[0.89, 0.15, 0.85, 0.07, ...]</i>
<i>"Best pizza recipe"</i>	→ <i>[0.02, 0.95, 0.10, 0.77, ...]</i>



3. How does a vector database store data?

```
{  
  id: 42,  
  vector: [0.91, 0.12, 0.88, ...],  
  text: "How to reset your password",  
  metadata: { category: "support", date: "2026-01-01" }  
}
```

The vector - Used for similarity search

The metadata / original content - So you can return the real answer and filter results

4. The Indexing Phase — getting data *ready* to search

This happens **once, ahead of time** (and again whenever you add new data).

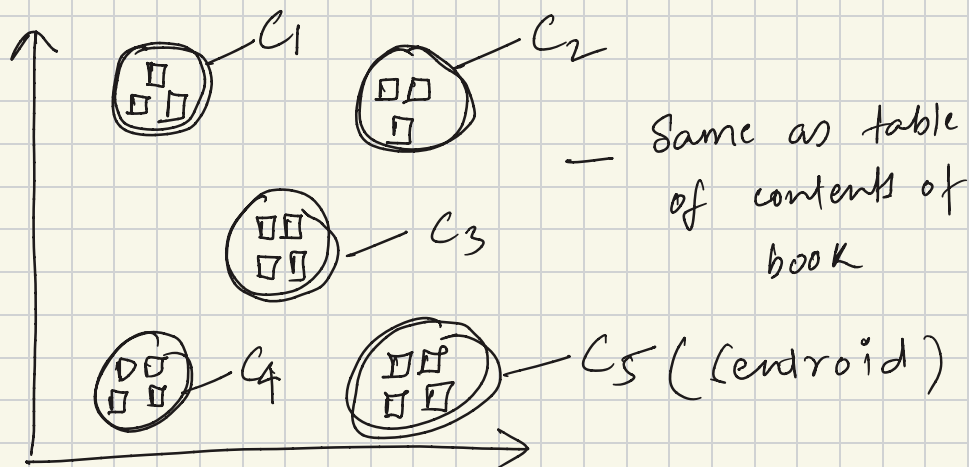
Chunk the Data → Embed each chunk →
store the vector → Build the index

Why do we need an "index"?

Problem --> If you have 10 million vectors and a user asks a question,

Compare the question's vector against **all 10 million** vectors, ?? --> **painfully slow**

The database **organizes the vectors cleverly in advance** --> This organization is the **index**.



Indexing Time (once)

10 M Vectors in N dimensional space

- 1000 natural center points (Buckets, centroid)
- Using K -means algorithm.
- Centroid 1 — holds $\sim 10,000$ vector.
- Centroid 2 — holds $\sim 10,000$ vector.
- $\vdots$
- Centroid 100 — holds $\sim 10,000$ vector.

→ Each one of the 10M vectors is assigned to closest centroid.

(5) Query Time

Someone does a query (search)

Round 1 → Compare Query Q
against the 1,000 centroids.

$$\text{distance}(Q, c_1) = 0.31$$

$$\text{distance}(Q, \underline{c_2}) = \underline{0.12} \leftarrow \text{closest}$$

$$\vdots$$
$$\text{distance}(Q, c_{1000}) = 0.77$$

The query belongs to Centroid
or Bucket c_2

Round 2 : Now search inside the
winning bucket.

Centroid 2 has $\sim 10,000$ vectors.

Compare query against all of
those 10,000 → Pick the top 5 closest.

Why it is so powerful ??

Instead of doing 10,000,000 comparisons which is comparing against each vector.

Round 1 : 1,000 comparisons

Round 2 : 10,000 comparisons.

Total : 11,000 comparisons only.

900x — less work.

ANN (Approximate Nearest Neighbour)

HNSW (Hierarchical Navigable Small World)

(uses the concept of graph)

RAG VS FINE TUNING

1. Frequency of Knowledge Updates

Fine Tuning: Best for static knowledge domains, such as legal, medical

RAG: Ideal for fast-changing knowledge environments, such as financial markets

2. Cost & Computational Requirements

Fine Tuning: Computationally expensive, requiring high-performance GPUs for model retraining.

RAG: More cost-effective, as it avoids the need for continuous model retraining. Instead, it relies on external knowledge bases, making it a scalable option for AI-driven companies.

3. Latency & Performance Constraints

Fine Tuning: Lower latency during inference, since all knowledge is embedded within the model,

RAG: Introduces retrieval delays, which can increase response time by 30-50% compared to fine tuned models.

4. Accuracy & Hallucination Risk

Fine Tuning: Reduces hallucination risks because the model internalizes facts rather than relying on external sources.

RAG: More susceptible to retrieval errors, leading to misleading or incorrect responses when retrieval quality is suboptimal.